

Medical Data Visualizer

DATA ANALYSIS WITH PYTHON PROJECT

This project involves visualizing and making calculations from medical examination data using `matplotlib`, `seaborn`, and `pandas`. The dataset values were collected during medical examinations and represent a real-world clinical dataset structure.

Core Stack & Libraries: `pandas` | `matplotlib` | `seaborn`

Data Description

The rows in the dataset represent patients and the columns represent information like body measurements, results from various blood tests, and lifestyle choices. You will use the dataset to explore the relationship between cardiac disease, body measurements, blood markers, and lifestyle choices.

File name: `medical_examination.csv`

Feature	Variable Type	Variable	Value Type
Age	Objective Feature	<code>age</code>	int (days)
Height	Objective Feature	<code>height</code>	int (cm)
Weight	Objective Feature	<code>weight</code>	float (kg)
Gender	Objective Feature	<code>gender</code>	categorical code
Systolic blood pressure	Examination Feature	<code>ap_hi</code>	int
Diastolic blood pressure	Examination Feature	<code>ap_lo</code>	int
Cholesterol	Examination Feature	<code>cholesterol</code>	1: normal, 2: above normal, 3: well above normal
Glucose	Examination Feature	<code>gluc</code>	1: normal, 2: above normal, 3: well above normal
Smoking	Subjective Feature	<code>smoke</code>	binary
Alcohol intake	Subjective Feature	<code>alco</code>	binary

Feature	Variable Type	Variable	Value Type
Physical activity	Subjective Feature	<code>active</code>	binary
Presence/absence of cardiovascular disease	Target Variable	<code>cardio</code>	binary

Instructions & Implementation Steps

By each number in your starter `medical_data_visualizer.py` file, add the code from the associated instruction number below:

1. Import the data from `medical_examination.csv` and assign it to the `df` variable.
2. Add an `overweight` column to the data. To determine if a person is overweight, first calculate their BMI by dividing their weight in kilograms by the square of their height in meters. If that value is > 25 then the person is overweight. Use the value `0` for NOT overweight and the value `1` for overweight.
3. Normalize data by making `0` always good and `1` always bad. If the value of `cholesterol` or `gluc` is 1, set the value to `0`. If the value is more than 1, set the value to `1`.
4. Draw the Categorical Plot in the `draw_cat_plot` function.
5. Create a DataFrame for the cat plot using `pd.melt` with values from `cholesterol`, `gluc`, `smoke`, `alco`, `active`, and `overweight` in the `df_cat` variable.
6. Group and reformat the data in `df_cat` to split it by `cardio`. Show the counts of each feature. You will have to rename one of the columns for the `catplot` to work correctly.
7. Convert the data into `long` format and create a chart that shows the value counts of the categorical features using the following method provided by the seaborn library import: `sns.catplot()`.
8. Get the figure for the output and store it in the `fig` variable.
9. Do not modify the next two lines of starter code.
10. Draw the Heat Map in the `draw_heat_map` function.
11. Clean the data in the `df_heat` variable by filtering out the following patient segments that represent incorrect data:
 - Diastolic pressure is higher than systolic (Keep the correct data with `df['ap_lo'] <= df['ap_hi']`).
 - Height is less than the 2.5th percentile (Keep the correct data with `df['height'] >= df['height'].quantile(0.025)`).
 - Height is more than the 97.5th percentile.
 - Weight is less than the 2.5th percentile.
 - Weight is more than the 97.5th percentile.
12. Calculate the correlation matrix and store it in the `corr` variable.

13. Generate a mask for the upper triangle and store it in the `mask` variable.
14. Set up the `matplotlib` figure.
15. Plot the correlation matrix using the method provided by the `seaborn` library import: `sns.heatmap()`.
16. Do not modify the next two lines of starter code.

Development & Testing

Development: Write your code in `medical_data_visualizer.py`. For development, you can use `main.py` to test your code.

Testing: The unit tests for this project are in `test_module.py`. The tests are automatically imported from `test_module.py` to `main.py` for your convenience.